

Lesson 2 Notes: Diagonal Matrix in C Using a Struct

1. Main idea of Lesson 2

Lesson 2 continues the same diagonal matrix idea from Lesson 1, but now we organize the data using a C `struct`.

In Lesson 1, we stored diagonal values in a simple 1D array.

In Lesson 2, we store two things together:

```
struct Matrix {  
    int A[10];  
    int n;  
};
```

This means:

- `A[10]` stores the diagonal values.
- `n` stores the size of the square matrix.

So instead of having only an array, we now have a complete matrix object-like structure that remembers both the stored values and the matrix dimension.

2. What we already know from Lesson 1

A diagonal matrix has important values only on the main diagonal.

Example:

```
5 0 0 0  
0 8 0 0  
0 0 9 0  
0 0 0 12
```

The diagonal positions are:

```
M[1,1]  
M[2,2]  
M[3,3]  
M[4,4]
```

The rule is:

```
i == j
```

If `i == j`, the position is on the diagonal.

If `i != j`, the position is not on the diagonal, so it is treated as zero.

The index formula is:

```
index = i - 1
```

This is because matrix positions start at `1`, but C arrays start at `0`.

3. Why Lesson 2 uses a struct

In Lesson 1, we used an array like this:

```
int A[10];
```

This stores values, but it does not store the matrix size.

For example, this array:

```
int A[10];
```

could be used for a `3 × 3`, `4 × 4`, or `5 × 5` diagonal matrix.

The array alone does not tell us the real matrix dimension.

So Lesson 2 uses a `struct`:

```
struct Matrix {  
    int A[10];  
    int n;  
};
```

Now the matrix stores:

A = diagonal values
n = matrix dimension

Example:

```
struct Matrix m;  
m.n = 4;
```

This means `m` represents a `4 × 4` matrix.

4. What is a struct?

A `struct` in C is used to group related data together.

For this lesson, the related data are:

1. The array that stores diagonal values.
2. The matrix dimension.

Code:

```
struct Matrix {  
    int A[10];  
    int n;  
};
```

Think of the struct as a box.

Inside the box:

A[10] stores the diagonal elements.
n stores the matrix size.

So a diagonal matrix is no longer just an array. It is a structured group of data.

5. Meaning of `A[10]`

The array:

```
int A[10];
```

can store up to 10 diagonal values.

This is a beginner-friendly fixed-size version.

If `n = 4`, we use only the first 4 positions:

```
A[0]  
A[1]  
A[2]  
A[3]
```

If `n = 5`, we use only the first 5 positions:

```
A[0]  
A[1]  
A[2]  
A[3]  
A[4]
```

The remaining positions are unused.

Example:

```
A[10] has space for 10 values.  
If n = 4, only 4 diagonal values are needed.
```

6. Meaning of `n`

The variable:

```
int n;
```

stores the dimension of the square matrix.

If:

```
m.n = 4;
```

then the matrix behaves like this:

```
4 × 4 matrix
```

Even though the program stores only 4 diagonal values, the visible matrix has 16 positions.

Example:

```
Stored array:  
A = [5, 8, 9, 12]
```

```
Visible matrix:  
5 0 0 0  
0 8 0 0  
0 0 9 0  
0 0 0 12
```

7. Compact storage example

Suppose the full matrix is:

```
5 0 0 0  
0 8 0 0  
0 0 9 0  
0 0 0 12
```

A normal 2D matrix would store all 16 values.

But a diagonal matrix stores only the diagonal values:

```
5, 8, 9, 12
```

So the compact array is:

```
A[0] = 5  
A[1] = 8  
A[2] = 9  
A[3] = 12
```

The struct stores:

```
A = [5, 8, 9, 12]  
n = 4
```

The zeros are not stored.

8. Mapping matrix positions to array positions

The diagonal positions map like this:

```
M[1,1] → A[0]  
M[2,2] → A[1]  
M[3,3] → A[2]  
M[4,4] → A[3]
```

The formula is:

```
index = i - 1
```

This formula is used only when:

```
i == j
```

If `i != j`, the value is not stored.

9. Why the formula is `i - 1`

Matrix positions in this lesson are written using 1-based indexing:

```
M[1,1]
M[2,2]
M[3,3]
```

But C arrays use 0-based indexing:

```
A[0]
A[1]
A[2]
```

So we subtract 1.

Example:

```
M[1,1] → A[1 - 1] → A[0]
M[2,2] → A[2 - 1] → A[1]
M[3,3] → A[3 - 1] → A[2]
```

That is why the formula is:

```
index = i - 1
```

10. The `set` function

The `set` function stores a value into the matrix.

Code:

```
void set(struct Matrix *m, int i, int j, int x) {
    if (i == j) {
        m->A[i - 1] = x;
    }
}
```

Meaning:

```
m = the matrix  
i = row number  
j = column number  
x = value to store
```

The function stores `x` only if the position is diagonal.

So it stores only when:

```
i == j
```

If the position is not diagonal, the function does nothing.

11. Why `set` receives a pointer

The `set` function must modify the real matrix.

That is why it receives:

```
struct Matrix *m
```

This means `m` is a pointer to a `struct Matrix`.

A pointer allows the function to change the original matrix from inside the function.

When we call `set`, we pass the address of the matrix:

```
set(&m, 2, 2, 8);
```

The `&m` means:

```
address of m
```

Because `set` receives the address, it can permanently change the matrix.

12. Why `set` uses `m->A`

Inside `set`, `m` is a pointer.

So we use the arrow operator:

```
m->A
```

Code:

```
m->A[i - 1] = x;
```

This means:

```
Go to the matrix pointed to by m.  
Access its array A.  
Store x at index i - 1.
```

Simple rule:

```
Normal struct variable: use dot  
Pointer to struct: use arrow
```

Example:

```
m.A      // use this when m is a normal struct variable  
m->A     // use this when m is a pointer to a struct
```

13. Dry run of `set` for a diagonal position

Call:

```
set(&m, 2, 2, 8);
```

Step 1: Identify the values.

```
i = 2  
j = 2  
x = 8
```

Step 2: Check the diagonal condition.

```
i == j  
2 == 2  
true
```

Step 3: Calculate the array index.

```
index = i - 1  
index = 2 - 1  
index = 1
```

Step 4: Store the value.

```
m->A[1] = 8;
```

So **8** is stored as the second diagonal value.

14. Dry run of **set** for a non-diagonal position

Call:

```
set(&m, 1, 2, 5);
```

Step 1: Identify the values.

```
i = 1  
j = 2  
x = 5
```

Step 2: Check the diagonal condition.

```
i == j  
1 == 2  
false
```

Step 3: Do nothing.

The value **5** is ignored.

Why?

Because (1,2) is not a diagonal position.

In a diagonal matrix, all non-diagonal positions must stay zero.

15. The `get` function

The `get` function returns a value from the matrix.

Code:

```
int get(struct Matrix m, int i, int j) {  
    if (i == j) {  
        return m.A[i - 1];  
    }  
    return 0;  
}
```

Meaning:

```
If i == j, return the stored diagonal value.  
If i != j, return 0.
```

So for diagonal positions, the function reads from the array.

For non-diagonal positions, the function returns zero immediately.

16. Why `get` does not need a pointer

The `get` function only reads the matrix.

It does not change anything.

So it can receive the matrix as a normal value:

```
struct Matrix m
```

This means `get` receives a copy of the struct.

That is okay because it only needs to read values.

Compare this with `set` :

```
void set(struct Matrix *m, int i, int j, int x)
```

`set` needs a pointer because it modifies the matrix.

But `get` does not need a pointer because it only reads.

17. Why `get` uses `m.A`

Inside `get`, `m` is not a pointer.

It is a normal struct variable.

So we use the dot operator:

```
m.A
```

Code:

```
return m.A[i - 1];
```

This means:

```
Access the array A inside the struct m.  
Return the value at index i - 1.
```

18. Dry run of `get` for a diagonal position

Assume:

```
A = [5, 8, 9, 12]
```

Call:

```
get(m, 3, 3);
```

Step 1: Check the condition.

```
i == j  
3 == 3  
true
```

Step 2: Calculate index.

```
index = i - 1  
index = 3 - 1  
index = 2
```

Step 3: Return the value.

```
return m.A[2];
```

Since:

```
m.A[2] = 9
```

The answer is:

```
9
```

19. Dry run of `get` for a non-diagonal position

Call:

```
get(m, 1, 4);
```

Step 1: Check the condition.

```
i == j  
1 == 4  
false
```

Step 2: Return zero.

```
return 0;
```

The function does not check the array.

It already knows `(1,4)` is not on the diagonal.

So the value must be zero.

20. Structural zeros

A structural zero is a zero that exists because of the matrix rule.

In a diagonal matrix, every non-diagonal position is a structural zero.

Example:

```
M[1,4] = 0  
M[2,3] = 0  
M[4,1] = 0
```

These zeros are not stored in the array.

The program simply returns or prints zero when needed.

This is why diagonal matrix storage saves memory.

21. The `display` function

The `display` function prints the full matrix.

Code:

```

void display(struct Matrix m) {
    int i, j;

    for (i = 1; i <= m.n; i++) {
        for (j = 1; j <= m.n; j++) {
            if (i == j) {
                printf("%d ", m.A[i - 1]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}

```

This function uses two loops:

Outer loop: controls rows
 Inner loop: controls columns

For every position `(i, j)`:

If `i == j`, print the stored value.
 If `i != j`, print 0.

22. Why display uses nested loops

The stored array contains only diagonal values.

But the displayed matrix must look like a full square matrix.

For a `4 × 4` matrix, display must print 16 positions:

row 1: 4 values
 row 2: 4 values
 row 3: 4 values
 row 4: 4 values

So it uses nested loops:

```
for (i = 1; i <= m.n; i++) {  
    for (j = 1; j <= m.n; j++) {  
        // print each position  
    }  
}
```

Even though storage is small, display must still print the full visible matrix.

23. Dry run of display

Assume:

```
A = [5, 8, 9, 12]  
n = 4
```

The function loops through all positions.

Row 1

```
(1,1): i == j, print A[0] = 5  
(1,2): i != j, print 0  
(1,3): i != j, print 0  
(1,4): i != j, print 0
```

Output row:

```
5 0 0 0
```

Row 2

```
(2,1): i != j, print 0  
(2,2): i == j, print A[1] = 8  
(2,3): i != j, print 0  
(2,4): i != j, print 0
```

Output row:

```
0 8 0 0
```

Row 3

```
(3,1): i != j, print 0  
(3,2): i != j, print 0  
(3,3): i == j, print A[2] = 9  
(3,4): i != j, print 0
```

Output row:

```
0 0 9 0
```

Row 4

```
(4,1): i != j, print 0  
(4,2): i != j, print 0  
(4,3): i != j, print 0  
(4,4): i == j, print A[3] = 12
```

Output row:

```
0 0 0 12
```

Full output:

```
5 0 0 0  
0 8 0 0  
0 0 9 0  
0 0 0 12
```

24. Full C program

```
#include <stdio.h>  
  
struct Matrix {  
    int A[10];  
    int n;  
};  
  
void set(struct Matrix *m, int i, int j, int x) {
```

```

        if (i == j) {
            m->A[i - 1] = x;
        }
    }

int get(struct Matrix m, int i, int j) {
    if (i == j) {
        return m.A[i - 1];
    }
    return 0;
}

void display(struct Matrix m) {
    int i, j;

    for (i = 1; i <= m.n; i++) {
        for (j = 1; j <= m.n; j++) {
            if (i == j) {
                printf("%d ", m.A[i - 1]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}

int main() {
    struct Matrix m = {{0}, 4};

    set(&m, 1, 1, 5);
    set(&m, 2, 2, 8);
    set(&m, 3, 3, 9);
    set(&m, 4, 4, 12);

    printf("Value at M[2][2]: %d\n\n", get(m, 2, 2));

    printf("Full matrix:\n");
    display(m);

    return 0;
}

```

25. Explanation of the full program

Header file

```
#include <stdio.h>
```

This is needed because the program uses `printf`.

Struct definition

```
struct Matrix {  
    int A[10];  
    int n;  
};
```

This creates a new data type named `struct Matrix`.

It contains:

```
A[10] = storage for diagonal values  
n = matrix dimension
```

Set function

```
void set(struct Matrix *m, int i, int j, int x)
```

This function modifies the matrix.

So it receives a pointer:

```
struct Matrix *m
```

It stores the value only if:

```
i == j
```

Get function

```
int get(struct Matrix m, int i, int j)
```

This function reads the matrix.

It returns:

```
stored value if i == j  
0 if i != j
```

Display function

```
void display(struct Matrix m)
```

This function prints the full matrix.

It prints real values on the diagonal and zeros everywhere else.

Main function

```
int main() {  
    struct Matrix m = {{0}, 4};
```

This creates a matrix named `m`.

The array is initialized with zeros.

The dimension is set to `4`.

Then these lines store diagonal values:

```
set(&m, 1, 1, 5);  
set(&m, 2, 2, 8);  
set(&m, 3, 3, 9);  
set(&m, 4, 4, 12);
```

Then this line gets the value at position `(2,2)`:

```
get(m, 2, 2)
```

Then this line displays the full matrix:

```
display(m);
```

26. Expected output

```
Value at M[2][2]: 8
```

```
Full matrix:
```

```
5 0 0 0  
0 8 0 0  
0 0 9 0  
0 0 0 12
```

27. Main trace

The program sets these values:

```
M[1,1] = 5  
M[2,2] = 8  
M[3,3] = 9  
M[4,4] = 12
```

These values are stored as:

```
A[0] = 5  
A[1] = 8  
A[2] = 9  
A[3] = 12
```

Then:

```
get(m, 2, 2)
```

returns:

```
A[1] = 8
```

Then `display(m)` prints:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

28. Difference between `m.A` and `m->A`

This is one of the most important ideas in Lesson 2.

Use `m.A` when `m` is a normal struct variable

Example:

```
struct Matrix m;
m.A[0] = 5;
m.n = 4;
```

Here, `m` is the actual struct variable.

So we use dot:

```
m.A
m.n
```

Use `m->A` when `m` is a pointer

Example:

```
void set(struct Matrix *m, int i, int j, int x)
```

Here, `m` is a pointer to a struct.

So we use arrow:

```
m->A  
m->n
```

29. Dot vs arrow summary

Situation	Operator	Example
Normal struct variable	.	m.A[i - 1]
Pointer to struct	->	m->A[i - 1]

Simple memory trick:

Dot means direct access.
Arrow means follow the pointer, then access.

30. Why `set(&m, ...)` uses `&`

The function call looks like this:

```
set(&m, 2, 2, 8);
```

The `&` symbol means address.

So `&m` means:

the address of matrix m

The `set` function expects a pointer:

```
struct Matrix *m
```

So we must pass the address:

```
&m
```

This allows `set` to modify the original matrix.

31. Why `get(m, ...)` does not use `&`

The function call looks like this:

```
get(m, 2, 2);
```

Here we pass `m`, not `&m`.

Why?

Because `get` is declared like this:

```
int get(struct Matrix m, int i, int j)
```

It expects a normal struct value, not a pointer.

Also, `get` only reads the matrix. It does not modify it.

So passing a copy is enough for this beginner version.

32. Complexity of Lesson 2 operations

Operation	Time Complexity	Space Complexity	Reason
Storage	-	<code>O(N)</code>	Only <code>N</code> diagonal values are stored.
<code>set()</code>	<code>O(1)</code>	-	It uses one condition and one index formula.
<code>get()</code>	<code>O(1)</code>	-	It uses one condition and possibly one array lookup.
<code>display()</code>	<code>O(N²)</code>	-	It prints every visible matrix position.

33. Why storage is `O(N)`

A full `N × N` matrix stores:

N^2 values

A diagonal matrix stores only:

N values

Example for `n = 4`:

Full matrix: $4 \times 4 = 16$ values
Diagonal storage: 4 values

So storage is:

$O(N)$

34. Why `set()` is $O(1)$

The `set` function does not loop.

It only does:

one condition check
one index calculation
one assignment if needed

So it takes constant time:

$O(1)$

35. Why `get()` is $O(1)$

The `get` function does not loop.

It only does:

one condition check
one possible array access

So it also takes constant time:

$O(1)$

36. Why `display()` is $O(N^2)$

The display function prints the full matrix.

For an $N \times N$ matrix, there are:

N^2 visible positions

Even though only N values are stored, display must print every row and every column.

So display takes:

$O(N^2)$

37. Beginner mental model

Think of the full matrix as what the user sees.

Think of the array as what the computer stores.

Visible matrix:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

Stored array:

```
A = [5, 8, 9, 12]
```

The struct remembers:

```
A = stored values  
n = matrix size
```

So the computer does not need to store the zeros.

It only creates them logically when needed.

38. Common beginner mistakes

Mistake 1: Using `m.A` inside `set`

Wrong:

```
void set(struct Matrix *m, int i, int j, int x) {  
    m.A[i - 1] = x;  
}
```

Correct:

```
void set(struct Matrix *m, int i, int j, int x) {  
    m->A[i - 1] = x;  
}
```

Reason:

```
Inside set, m is a pointer.  
Pointers use ->, not .
```

Mistake 2: Forgetting `&m` when calling `set`

Wrong:

```
set(m, 2, 2, 8);
```

Correct:

```
set(&m, 2, 2, 8);
```

Reason:

```
set expects a pointer, so we pass the address of m.
```

Mistake 3: Using `&m` when calling `get`

Wrong for this lesson's version:

```
get(&m, 2, 2);
```

Correct:

```
get(m, 2, 2);
```

Reason:

```
get expects a normal struct value, not a pointer.
```

Mistake 4: Using `A[i]` instead of `A[i - 1]`

Wrong:

```
m->A[i] = x;
```

Correct:

```
m->A[i - 1] = x;
```

Reason:

Matrix positions start at 1.
C array indexes start at 0.

Mistake 5: Storing non-diagonal values

Wrong idea:

If the user gives $M[1,2] = 7$, store it somewhere.

Correct idea:

Ignore it because $M[1,2]$ must be zero in a diagonal matrix.

Mistake 6: Thinking display means the zeros are stored

The displayed matrix may look like this:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

But the stored array is only:

$A = [5, 8, 9, 12]$

The zeros are printed, not stored.

39. Practice example 1

Suppose:

$A = [4, 6, 10, 15]$
 $n = 4$

This represents:

```
4 0 0 0
0 6 0 0
0 0 10 0
0 0 0 15
```

Questions

```
get(m, 1, 1)
```

Answer:

```
4
```

Reason:

```
1 == 1, so return A[0].
```

```
get(m, 3, 3)
```

Answer:

```
10
```

Reason:

```
3 == 3, so return A[2].
```

```
get(m, 2, 4)
```

Answer:

```
0
```

Reason:

`2 != 4`, so the position is non-diagonal.

`set(&m, 4, 4, 20)`

Result:

`A[3] = 20`

Reason:

`4 == 4`, so the value is stored.

`set(&m, 1, 3, 99)`

Result:

Nothing changes.

Reason:

`1 != 3`, so the position is non-diagonal.

40. Practice example 2

Suppose:

`n = 5`
`A = [2, 4, 6, 8, 10]`

Full matrix:

```

2 0 0 0 0
0 4 0 0 0
0 0 6 0 0
0 0 0 8 0
0 0 0 0 10

```

Questions and answers

Operation	Answer	Reason
<code>get(m, 1, 1)</code>	2	Diagonal position, return <code>A[0]</code> .
<code>get(m, 5, 5)</code>	10	Diagonal position, return <code>A[4]</code> .
<code>get(m, 3, 4)</code>	0	Non-diagonal position.
<code>get(m, 5, 2)</code>	0	Non-diagonal position.
<code>set(&m, 3, 3, 99)</code>	stores 99 at <code>A[2]</code>	Diagonal position.
<code>set(&m, 2, 5, 77)</code>	ignored	Non-diagonal position.

41. Lesson 2 key rules

Struct design

```

struct Matrix {
    int A[10];
    int n;
};

```

Diagonal condition

```
i == j
```

Diagonal index formula

```
index = i - 1
```

Set rule

```
If i == j, store x in A[i - 1].  
If i != j, do nothing.
```

Get rule

```
If i == j, return A[i - 1].  
If i != j, return 0.
```

Display rule

```
Loop through all positions.  
Print A[i - 1] when i == j.  
Print 0 when i != j.
```

Pointer rule

```
set modifies the matrix, so it uses a pointer.  
get only reads the matrix, so it can use a value copy.
```

Dot and arrow rule

```
m.A means direct struct access.  
m->A means pointer-to-struct access.
```

42. Lesson 2 final summary

Lesson 2 teaches how to implement a diagonal matrix in C using a `struct`.

The diagonal matrix still follows the same rule:

```
Only positions where i == j are stored.
```

All other positions are structural zeros.

The struct stores both the compact array and the matrix size:

```
struct Matrix {  
    int A[10];  
    int n;  
};
```

The `set` function stores values only on the diagonal:

```
void set(struct Matrix *m, int i, int j, int x)
```

It uses a pointer because it modifies the real matrix.

The `get` function returns stored values or zero:

```
int get(struct Matrix m, int i, int j)
```

It does not need a pointer because it only reads.

The `display` function prints the full matrix using nested loops.

It prints real diagonal values and fake zeros for non-diagonal positions.

The most important programming idea in this lesson is the difference between:

```
m.A
```

and:

```
m->A
```

Use `m.A` when `m` is a normal struct variable.

Use `m->A` when `m` is a pointer to a struct.

43. Exit questions

You should be able to answer these before moving to Lesson 3.

1. Why do we use a `struct` instead of only using an array?
2. What does `A[10]` store?

3. What does `n` store?
 4. Why does `set` use `struct Matrix *m`?
 5. Why can `get` use `struct Matrix m`?
 6. What is the difference between `m.A` and `m->A`?
 7. Why does `set(&m, 1, 2, 5)` do nothing?
 8. Why does `get(m, 1, 4)` return `0`?
 9. Why does display take $O(N^2)$ time?
 10. What is the index formula for a diagonal matrix?
-

44. Short answers to exit questions

1. We use a `struct` so the array and matrix size stay together.
2. `A[10]` stores the diagonal values.
3. `n` stores the dimension of the square matrix.
4. `set` uses a pointer because it changes the original matrix.
5. `get` can use a value copy because it only reads.
6. `m.A` is used with a normal struct variable; `m->A` is used with a pointer to a struct.
7. `set(&m, 1, 2, 5)` does nothing because `1 != 2`, so the position is not diagonal.
8. `get(m, 1, 4)` returns `0` because `(1,4)` is a structural zero.
9. Display takes $O(N^2)$ because it prints every visible matrix position.
10. The index formula is `i - 1`.